Développement Web

Antoine TIREL

Second trimestre 2026

Table des matières

1 Une transition client **2**
1.1 Client 2
1.1.1 Navigateur 2
1.1.2 Le DOM 3

2 Une plongée en profondeur **5**
2.1 Nouveaux "langages" 5
2.1.1 HTML 5
2.1.2 CSS 5
2.1.3 JavaScript 7

3 JavaScript en profondeur **8**
3.1 Quelques notions de base 8
3.1.1 L'égalité 8
3.1.2 null et undefined 9
3.1.3 Les variables 9
3.2 Un peu d'histoire 9
3.3 Aujourd'hui 10
3.4 Asynchrone vs Synchrone 10
3.5 Observables 10
3.6 Debug 11
3.7 Node 11

1 Une transition client

Dans le cours, jusqu'à présent, nous nous sommes focalisés sur la partie serveur. Nous avons vu la différence entre les responsabilités dans un serveur et le travail réalisé par chaque partie.
Pour ce faire, nous avons surtout utilisé l'interface Swagger, et les plus motivés d'entre vous ~~ou les plus fous~~ avec wget, directement en ligne de commande.
Mais vous l'avez sans doute remarqué, les sites web que vous parcourez tous les jours ne ressemblent pas à des terminaux, donc comment ça marche ?

1.1 Client

On a vu qu'un client prenait le plus souvent la forme d'un navigateur internet, qui interprétait et affichait ce que lui renvoyait un serveur avec de la mise en forme, mais concrètement, ça donne quoi ?

1.1.1 Navigateur

Un navigateur internet permet de parcourir le web et d'afficher des pages HTML. Pour ce faire, le navigateur va procéder à différentes transformations et interpréter le code, en très large majorité du JavaScript, du HTML et du CSS.

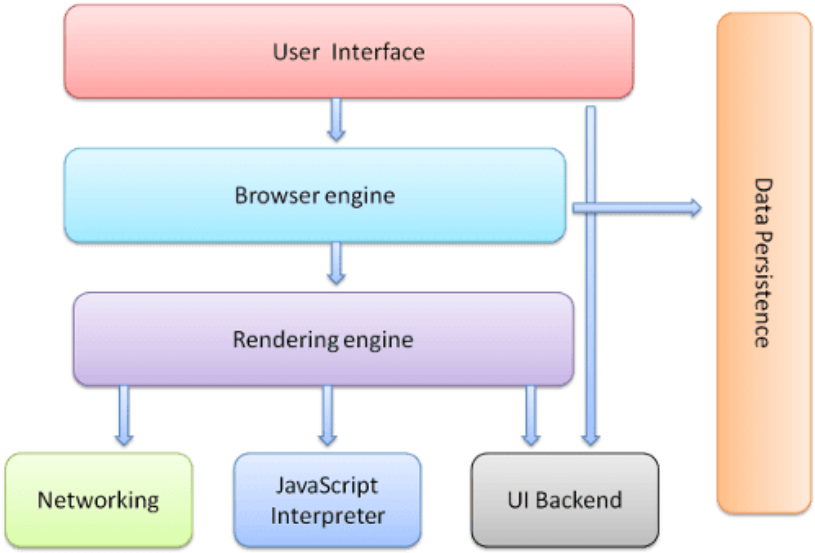
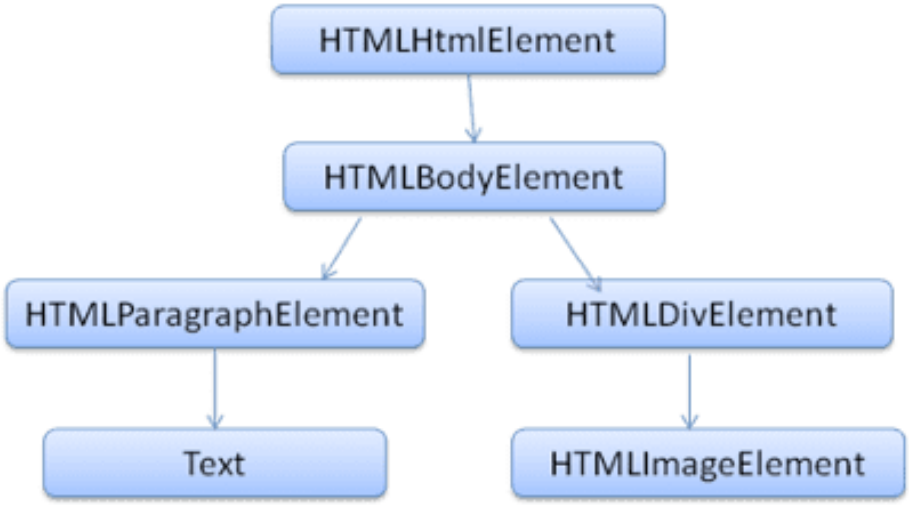


Figure 1: Browser components

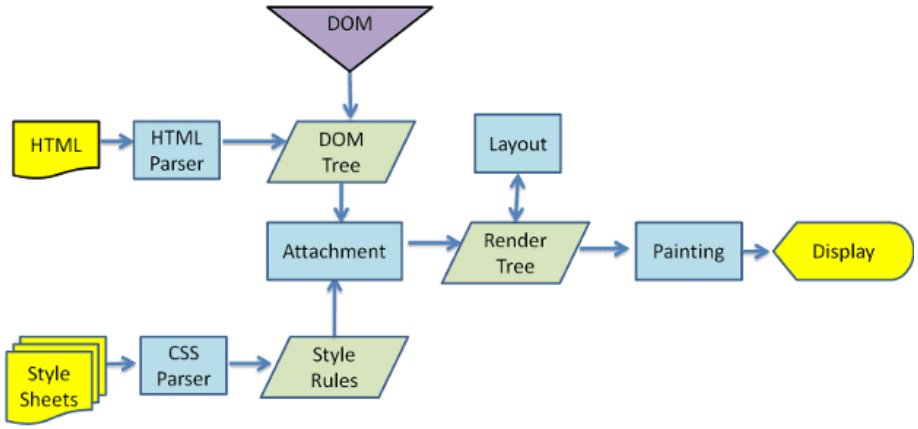
1.1.2 Le DOM

Le DOM ou Document Object Model est un modèle de représentation qui prend la forme d'un arbre, dont on retrouve généralement la structure dans le code source de la page ctrl U ou F12 pour les intimes.

```
<html>
  <body>
    <p>
      Hello World
    </p>
    <div> <img src="example.png"/></div>
  </body>
</html>
```



Lors du rendu d'un site, le navigateur va parcourir le code HTML, puis s'en servir pour générer le DOM comme ci-dessus.
Puis, le navigateur parcourt le CSS pour définir ce que l'on appelle les règles de style.
L'arbre du DOM est parcouru avec application des règles de style, pour obtenir un arbre de rendu, sur lequel on vient ajouter du code JavaScript, principalement pour tout ce qui va toucher aux interactions.



En direct, ça donne un truc comme ça pour Firefox, avec son outil Gecko.

2 Une plongée en profondeur

2.1 Nouveaux "langages"

2.1.1 HTML

Le HTML ou HyperText Markup Language, est à la racine de tout site Web, même à la racine du Web.
Le HTML n'est généralement pas considéré comme un langage de programmation, mais plus de balisage, comme le XML ou JSON.
Le HTML suit une énorme qui est éternellement en cours d'évolution (dernière mise à jour le 31 mars 2026).
Pour faire du HTML, on peut utiliser plusieurs outils en ligne comme StackBlitz ou Codepen.

2.1.2 CSS

Le CSS, ou Cascading Style Sheet, permet de définir un ensemble de règles qui vont définir comment doit apparaître tout ce qui est à l'écran.
On intègre le CSS directement dans le code HTML, par l'intermédiaire d'une référence.

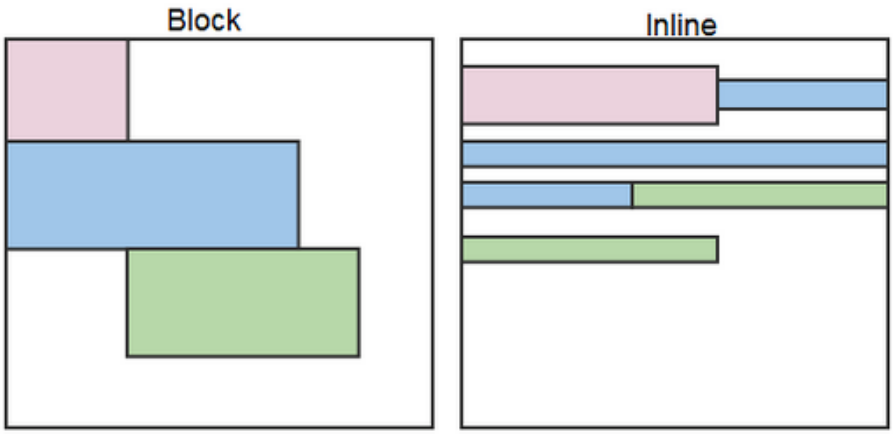
```
<html>
  <head>
    <link rel="stylesheet" href="styles.css">
  </head>
  <body>
    <p>
      Hello World
    </p>
    <div> <img src="example.png"/></div>
  </body>
</html>
```

Ou directement dans le code sur lequel on souhaite appliquer la règle.

```
<html>
  <body>
    <p>
      Hello World
    </p>
    <div style="text-align: center"> <img src="example.png"/></div>
  </body>
</html>
```

De manière générale, on utilise des sélecteurs dans les fichiers CSS pour définir ce sur quoi ils vont s'appliquer, par l'intermédiaire de l'attribut class.

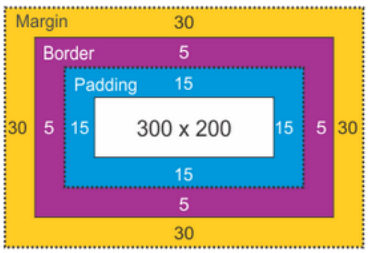
Le CSS possède plusieurs attributs qui permettent de définir le comportement des morceaux de code HTML auxquels ils sont reliés.
Le flux (ou flow) détermine la manière de parcourir la page, les deux flux les plus courants sont le block, qui propose une lecture verticale, et le inline, qui propose une lecture horizontale



Il est également important de définir le modèle de boite, entre les deux options principales :

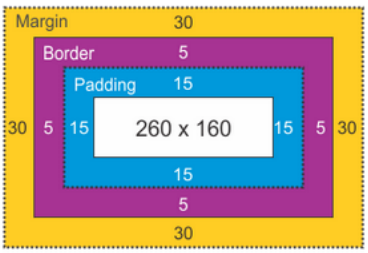
- content-box qui fixe la taille du bloc rendu à celle de son contenu
- border-box qui fixe la taille du bloc rendu à celle de son contenu ET des margin et padding qui s'appliquent dessus.

Box Model is content-box




```
div{
      width: 300px;
      height: 200px;
      padding: 15px;
      border: 5px solid grey;
      margin: 30px;
      -moz-box-sizing: content-box;
      -webkit-box-sizing: content-box;
      box-sizing: content-box;
    }
```

Box Model is border-box




```
div{
      width: 300px;
      height: 200px;
      padding: 15px;
      border: 5px solid grey;
      margin: 30px;
      -moz-box-sizing: border-box;
      -webkit-box-sizing: border-box;
      box-sizing: border-box;
    }
```

Le CSS est défini de manière globale, ce qui signifie qu'il faut être particulièrement prudent lors de l'import de multiples fichiers CSS : si les deux définissent des règles distinctes s'appliquant sur un même bloc, le rendu pourrait être très aléatoire.
Il est très important de définir les sélecteurs de la manière la plus précise et spécifique possible pour limiter au maximum ce genre de conflit.
On utilisera généralement des noms de classe particulièrement spécifiques au bloc que l'on manipule pour s'en assurer.

2.1.3 JavaScript

Attention au piège !

Le code JavaScript se place juste avant la balise de clôture du body, avec l'aide des balises <script> </script>
Au début, avec JavaScript, on faisait des trucs comme faire tomber de la neige, il y a dix ans, on s'est dit que ça pouvait aussi servir pour jouer de la musique d'ascenseur
JavaScript est le langage possédant le plus d'utilisation, le plus de repositories Git, le plus grand nombre de commits sur GitHub et GitLab.
JavaScript est, comme son nom l'indique, un langage de script, non typé (enfin presque pas), avec lequel il est donc possible de faire n'importe quoi.
Il a été écrit en une semaine par un monsieur (Brendan Eich) qui s'ennuyait un peu trop chez Netscape, et qui a voulu faire un langage de script pour augmenter l'interactivité entre les différents blocs HTML.

Le langage permet d'accéder aux propriétés window (qui définit les interactions avec la page dans son ensemble) et au DOM pour interagir directement avec.
Il permet de gérer aussi bien des exécutions synchrones qu'asynchrones.
Le langage possède malheureusement de nombreux défauts :

- Différences d'implémentation suivant les évolutions de la spécification
- Les performances des bibliothèques sont assez inégales
- Le support sur les différents navigateurs demande du temps et de l'argent

Pour ne citer que de nouveaux écueils :
Une petite liste des légendes de JavaScript :

- typeof NaN donne number
- typeof [] donne object
- 1 + "2" donne "12"
- 1 - "2" donne -1
- true vaut 1 donc true + true = 2

3 JavaScript en profondeur

3.1 Quelques notions de base

3.1.1 L'égalité

En JavaScript, il existe un nouveau concept d'égalité, l'égalité faible, ou égalité abstraite, qui s'écrit

$$==$$

L'égalité plus classique, ou égalité stricte s'écrit

$$===$$

Pourquoi cette différence est importante et pourquoi faut il TOUJOURS utiliser l'égalité stricte ?

$$[\,] == 0$$

$$0 == "0"$$

$$[\,] ==![\,]$$

$$[\,] == false$$

$$null == undefined$$

À l'inverse :

$$[\,] \,! == 0$$

$$0 \,! == "0"$$

$$[\,] \,! ==![\,]$$

$$[\,] \,! == false$$

$$null! == undefined$$

3.1.2 null et undefined

De part son aspect "tout est une string", JavaScript traite les objets comme des JSON en série, et utilise un système de clé-valeur pour les parcourir.
Il introduit donc un nouveau concept que n'ont pas la plupart des autres langages, le type undefined.
Il se distingue du type "null" uniquement d'un point de vue conceptuel :
Si un attribut vaut null, cela veut dire qu'il peut avoir une valeur, ou ne pas en avoir.
Un attribut ~~ne peut pas~~ ne devrait pas pouvoir avoir la valeur undefined, si un attribut vaut undefined, cela signifie dans l'absolu : la ressource que tu me demandes n'existe pas et n'existera jamais, cherche ailleurs ou autre chose.

3.1.3 Les variables

Vous pourrez souvent être amenés à manipuler des variables en JavaScript. Dans le langage, il existe trois façons de définir une variable :

- var
- let
- const

La première permet de déclarer une variable globale, peu importe son lieu d'utilisation. C'est à dire qu'il est possible d'écraser une var n'importe où dans le code en écrivant tout simplement var myVar = newValue. Elle n'est plus utilisée depuis 2015 et elle est à proscrire.
La deuxième permet de déclarer une variable locale, dans son scope.
La troisième permet de déclarer une constante locale, dans son scope.

3.2 Un peu d'histoire

Au début, on faisait essentiellement du JQuery.
C'était une période simple, où on faisait des choses simples, en bonne intelligence.
Le problème, c'est que la trop grande simplicité des interactions possibles, à savoir uniquement des queries, a rapidement conduit à la production de code spaghetti.

```
10 i = 0
20 i = i + 1
30 IF i <> 11 THEN GOTO 80
40 IF i = 11 THEN GOTO 60
50 GOTO 20
60 PRINT "Programme␣terminé."
70 END
80 PRINT i & "␣au␣carré␣=␣" & i * i
90 GOTO 20
```

Pour faire évoluer ça, on a décidé de définir une structure, que l'on a déjà vue dans le cours, il s'agit de la structure MVC.

On transforme ainsi une page Web enrichie, c'est à dire un bloc HTML avec quelques scripts listés de manière simple à une vraie application Web, ce qui a donné lieu à une période un peu mystique, où tout le monde cherchait à devenir l'application qui allait dominer la suite du marché, ou celle qui allait renverser le marché existant.

3.3 Aujourd'hui

Aujourd'hui, JavaScript est passé à travers une vraie phase d'industrialisation, avec un outillage complexe, beaucoup d'automatisation, qui permet de gagner du temps, de limiter les erreurs...
On peut jouer avec des modules, qui permettent de grouper différentes application JavaScript que l'on va manipuler dans leur ensemble.

3.4 Asynchrone vs Synchrone

Une exécution synchrone est une exécution simple, chaque ligne de code est exécutée à la suite de l'exécution de la précédente et de sa complétion.
Une exécution asynchrone lance toutes les étapes dans l'ordre des lignes de code, mais n'attend pas la complétion de la précédente pour lancer la suivante.
En JavaScript, on utilise la notion de promesses pour gérer l'asynchrone.

```javascript
/* CONTEXT (this) */

class MyClass {
  value = 42;

  method1() {
    return this.value;
  }

  method2 = () => {
    return this.value;
  }
}

const instance = new MyClass();

instance.method1(); // 42
instance.method2(); // 42

const { method1, method2 } = instance;

method1(); // error
method2(); // 42
```

```javascript
/* ASYNCHRONICITY */

const promise1 = Promise.resolve();
const promise2 = Promise.reject();

promise1
  .then(() => console.log('ok 1'))
  .catch(() => console.log('error 1'))
  .finally(() => console.log('finally 1'));

console.log('after promise 1');

promise2
  .then(value => console.log('ok 2'))
  .catch(error => console.log('error 2'))
  .finally(() => console.log('finally 2'));

console.log('after promise 2');

// after promise 1
// after promise 2
// ok 1
// finally 1
// error 2
// finally 2
```

3.5 Observables

Les observables permettent de gérer des flux de données asynchrones, ils sont particulièrement utilisé par RXJS qui fournit une grande quantité d'opérateurs.
On peut voir un observable comme une évolution de la promesse native : la où la promesse ne permet que de recevoir un résultat unique et de le traiter suivant

un cas de succès ou d'erreur, un observable permet de réaliser de nombreuses opérations intermédiaires.

3.6 Debug

La plupart des devtools (commande qui apparaît quand vous appuyez sur F12, ou Ctrl + Maj + i ou Cmd + Maj + i sur un navigateur incluent un onglet de debug qui vous permet de parcourir votre application, d'accéder au DOM, à la définition des styles...
N'oubliez jamais : tout ce qui peut être modifié est écrasé au moindre rafraîchissement, mais peut entraîner une modification de la page à la volée.

3.7 Node

Tous les systèmes de JavaScript avec lesquels nous allons être amenés à jouer sont basés sur NodeJS.
Node.js est un projet open-source créé en 2009 qui permet d'exécuter du code JavaScript directement dans un terminal.
Node.js permet également de créer des serveurs http, donc ce que l'on a fait en première partie de cours.
Il possède l'avantage majeur de centraliser tout sur du JavaScript, qui doit dans tous les cas être utilisé pour le développement front-end, donc du client.
Node.js a eu un succès monstrueux, et est aujourd'hui au cur de projets Microsoft, Apple, LinkedIn...
Node.js est installé directement avec son gestionnaire de paquets, npm ou node package manager, qui permet d'installer à la volée n'importe quel projet, n'importe quelle image et de mettre à jour le système de dépendances.
Pour gérer ses dépendances, node utilise un fichier appelé package.json, qui liste les package dont l'application a besoin pour tourner, comme dans l'exemple ci-après :

```
{
  "private": true,
  "name": "CHANGE ME!",
  "version": "0.1.0",
  "scripts": {
    "start": "ng serve",
    "build": "ng build"
  },
  "dependencies": {
    "@angular/core": "^7.2.2",
    "@ionic/angular": "^4.0.0"
  },
  "devDependencies": {
    "@angular/cli": "~7.2.3",
    "typescript": "~3.1.6"
  }
}
```